

# Relatório de Avaliação Independente

## Solver de Difusão de Nêutrons Monoenergética 1D/2D

(Opções 1 e 3 do projeto final de Física de Reatores)

Bateria de testes de verificação, física, consistência numérica e robustez

6 de julho de 2026

### Resumo

Este relatório documenta uma bateria independente de 25 verificações aplicadas ao solver `difusao` (volumes finitos *cell-centered*, LU esparsa/CG, *power iteration*/Wielandt), organizada em quatro frentes: (A) verificação contra soluções analíticas, (B) física de reatores, (C) consistência numérica e (D) robustez de entrada. Resultado global: **a física e a numérica do código estão sólidas** — 18/18 testes da suíte própria passam, os 10 casos obrigatórios conservam nêutrons com resíduo  $\leq 3,5 \times 10^{-13}$ , o autovalor converge para o benchmark analítico com erro  $8,5 \times 10^{-12}$  após extrapolação de Richardson, e todas as invariâncias físicas são satisfeitas em precisão de máquina. Foram identificadas **5 falhas de robustez na camada de entrada** (tratamento de erros e avisos ao usuário), nenhuma delas afetando a correção dos resultados numéricos. As cinco foram **corrigidas** com alterações localizadas em `input_data.py` e `malha.py` (Seção 7); após as correções, a bateria completa e a suíte própria (18/18) passam integralmente. Todos os números deste relatório são reproduzíveis via `relatorio/testes_professor.py`.

## 1 Objeto avaliado e metodologia

O código resolve a equação de difusão monoenergética estacionária

$$-\nabla \cdot (D \nabla \phi) + \Sigma_a \phi = Q \quad \text{ou} \quad \frac{1}{k_{\text{eff}}} \nu \Sigma_f \phi, \quad D = \frac{1}{3\Sigma_t}, \quad (1)$$

em meios heterogêneos 1D/2D, por volumes finitos *cell-centered* com média harmônica de  $D$  nas interfaces, condição de vácuo de Marshak ( $\alpha = 2D/(h+4D)$ ) e reflexão por corrente nula. Sistemas lineares por LU esparsa (`splu`, MMD) ou CG+Jacobi; autovalor por *power iteration* ou Wielandt em dois estágios.

A avaliação exercita o código *sem alterá-lo*: os módulos são importados diretamente (testes A–C) ou o executável `main.py` é invocado como o usuário final o faria (testes D). Critérios de aprovação são declarados *a priori* em cada teste.

### Ambiente

Python 3.14.5 (venv), `numpy/scipy/pyyaml` conforme `requirements.txt`, Linux x86-64. **Observação de reprodutibilidade:** o interpretador do sistema não possuía as dependências instaladas; foi necessário criar um ambiente virtual. O README instrui corretamente o `pip install`, mas recomenda-se declarar explicitamente a criação do venv.

## 2 Suíte de testes própria e casos obrigatórios

A suíte `pytest` do próprio projeto foi executada integralmente: **18/18 testes passam** em 0,3 s. Em seguida os 10 casos obrigatórios do enunciado (placa de 20 cm,  $\Sigma_t = 1$ , vácuo em ambos os lados):

| Caso | $\Sigma_{s1}$ | $\Sigma_{s2}$ | Absorção | Fuga     | Resíduo do balanço     |
|------|---------------|---------------|----------|----------|------------------------|
| 01   | 0,8           | 0,8           | 9,148749 | 0,851251 | $2,03 \times 10^{-14}$ |
| 02   | 0,8           | 0,9*          | 9,072901 | 0,927099 | $2,04 \times 10^{-14}$ |
| 03   | 0,8           | 0,99          | 8,668636 | 1,331364 | $1,03 \times 10^{-14}$ |
| 04   | 0,8           | 0,999*        | 8,525411 | 1,474589 | $7,64 \times 10^{-15}$ |
| 05   | 0,9           | —             | 6,155659 | 3,844341 | $1,41 \times 10^{-13}$ |
| 06   | 0,99          | —             | 5,107640 | 4,892360 | $1,60 \times 10^{-13}$ |
| 07   | 0,99          | —             | 4,930043 | 5,069957 | $1,41 \times 10^{-13}$ |
| 08   | 0,999         | —             | 2,835959 | 7,164041 | $1,92 \times 10^{-13}$ |
| 09   | 0,9999        | —             | 4,756272 | 5,243728 | $2,36 \times 10^{-13}$ |
| 10   | 0,9999        | —             | 2,464999 | 7,535001 | $3,50 \times 10^{-13}$ |

Tabela 1: Balanço de nêutrons dos 10 casos obrigatórios (fonte total = 10 em todos). Combinações conforme os arquivos `inputs/caso01-10.yaml` (colunas  $\Sigma_s$  meramente indicativas; ver os arquivos para a definição completa). Todos os resíduos ficam 5–6 ordens de grandeza abaixo do limiar de aceitação do próprio código ( $10^{-8}$ ). **APROVADO**

Os exemplos adicionais também executam corretamente: 2D com LU (resíduo  $7,4 \times 10^{-14}$ ), 2D com CG+Jacobi (297 iterações, resíduo  $1,8 \times 10^{-12}$ ) e autovalor Wielandt ( $k_{\text{eff}} = 1,27764735$ , 34 iterações, 3 fatorações LU, resíduo  $2,9 \times 10^{-16}$ ).

### 3 Frente A — Verificação contra soluções analíticas

#### 3.1 A1. Meio infinito homogêneo

Reflexão em ambos os lados com fonte uniforme deve reproduzir  $\phi = Q/\Sigma_a = 2,0$  exatamente. Erro relativo máximo obtido:  $2,5 \times 10^{-14}$  (precisão de máquina). **APROVADO**

#### 3.2 A2. Ordem de convergência espacial (fonte fixa)

Meia-placa (reflexiva-vácuo) contra a solução analítica  $\phi(x) = \frac{Q}{\Sigma_a} [1 - \cosh(x/L_d)/\text{den}]$ :

| $N$         | 40                    | 80                    | 160                   | 320                   | 640                   |
|-------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|
| erro max    | $1,56 \times 10^{-3}$ | $4,17 \times 10^{-4}$ | $1,08 \times 10^{-4}$ | $2,73 \times 10^{-5}$ | $6,88 \times 10^{-6}$ |
| ordem local | —                     | 1,907                 | 1,955                 | 1,978                 | 1,989                 |

A ordem tende monotonicamente a 2, como esperado para o esquema. **APROVADO**

#### 3.3 A3. Placa nua crítica (benchmark analítico de autovalor)

**Este teste não existia na suíte do projeto** — a suíte original só verificava Wielandt  $\equiv$  *power* (consistência interna, não verificação). Para a placa nua ( $L = 20$  cm,  $\Sigma_t = 1$ ,  $\Sigma_s = 0,9$ ,  $\nu\Sigma_f = 0,15$ ) com condição de Marshak, o modo fundamental satisfaz

$$\tan\left(\frac{BL}{2}\right) = \frac{1}{2DB}, \quad k_{\text{eff}}^{\text{ana}} = \frac{\nu\Sigma_f}{\Sigma_a + DB^2}, \quad (2)$$

cuja raiz dá  $B = 0,14729157 \text{ cm}^{-1}$  e  $k_{\text{eff}}^{\text{ana}} = 1,3988413535$ .

| $N$                                              | 50                    | 100                   | 200                   | 400                   | 800                   |
|--------------------------------------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|
| $ k_{\text{eff}} - k_{\text{eff}}^{\text{ana}} $ | $3,07 \times 10^{-5}$ | $7,67 \times 10^{-6}$ | $1,92 \times 10^{-6}$ | $4,79 \times 10^{-7}$ | $1,20 \times 10^{-7}$ |
| ordem local                                      | —                     | 2,000                 | 2,000                 | 2,000                 | 2,000                 |

Convergência de ordem 2 impecável. A extrapolação de Richardson das duas malhas mais finas dá  $k_{\text{eff}} = 1,3988413535$ , com erro de  $8,5 \times 10^{-12}$  contra o valor analítico. O acoplamento fissão–fuga–absorção está correto. **APROVADO**

## 4 Frente B — Física de reatores

### 4.1 B1. Continuidade da corrente na interface de material

No caso  $\Sigma_{s1} = 0,99 \rightarrow \Sigma_{s2} = 0,8$  (interface em  $x = 10$  cm), a corrente reconstruída pelos dois lados da interface:  $J^- = J^+ = 3,39206395$ , salto relativo  $1,4 \times 10^{-14}$ . A média harmônica garante  $J^- = J^+$  por construção; a verificação numérica confirma a implementação. **APROVADO**

### 4.2 B2. Sensibilidade de $k_{\text{eff}}$ (teoria de perturbação)

Em um grupo, multiplicar  $\nu\Sigma_f$  por 1,01 deve escalar  $k_{\text{eff}}$  *exatamente* por 1,01 (mesmo fluxo, operador de fissão escalado). Obtido: razão = 1,0100000000, desvio  $8,4 \times 10^{-14}$ . Endurecer o refletor ( $\Sigma_s$ : 0,95  $\rightarrow$  0,90, i.e.  $\Sigma_a$  maior) reduz  $k_{\text{eff}}$  de 1,277574 para 1,274863 — sinal físico correto. **APROVADO**

### 4.3 B3. Efeito do refletor

Núcleo nu de 20 cm ( $k = 1,269401$ ) versus o mesmo núcleo com 20 cm de refletor de cada lado ( $k = 1,277574$ ): ganho de reatividade de +817 pcm e achatamento do fluxo no núcleo (fator de pico 1,480  $\rightarrow$  1,321). Ambos os efeitos na direção fisicamente correta. **APROVADO**

### 4.4 B4. Razão de dominância e Wielandt

Dois núcleos físeis fracamente acoplados (razão de dominância  $\approx 0,9997$ ): *power* converge em 128 iterações, Wielandt em 34 (3 fatorações LU), ganho de 3,8 $\times$ , com  $|k_W - k_P| = 3,0 \times 10^{-9}$ . O Wielandt entrega o mesmo modo fundamental com custo muito menor exatamente no regime em que a *power iteration* sofre. **APROVADO**

### 4.5 B5. Quantificação do limite difusivo (quantitativo)

O código usa distância extrapolada de Marshak  $d = 2D = 0,6667$  cm; a teoria de transporte dá  $d = 0,7104 \lambda_{tr} = 0,7104$  cm. Comparando as soluções analíticas de meia-placa com cada  $d$ :

|                                       | $c = 0,8$ | $c = 0,9999$ |
|---------------------------------------|-----------|--------------|
| diferença relativa máxima em $\phi$   | 1,44 %    | 0,77 %       |
| diferença relativa em $\phi$ na borda | 4,06 %    | 6,14 %       |

Ou seja: a limitação declarada no README é real, mas *quantificada*: o erro de modelo concentra-se na vizinhança da borda de vácuo ( $\sim 4$ –6 % no fluxo de borda) e é  $\lesssim 1,5$  % no interior. É limitação da aproximação de difusão, não defeito de implementação.

## 5 Frente C — Consistência numérica

### 5.1 C1. Pior caso combinado: 2D + malha graduada + autovalor

Configuração que nenhum teste da suíte original cobria simultaneamente: domínio 2D  $40 \times 40$  cm, graduação  $p_x = 1,8$ ,  $p_y = 1,4$ , núcleo central fissil, BC mista (3 vácuos + 1 reflexiva). Resultado:  $k_{\text{eff}} = 1,266363$ , **resíduo de balanço**  $1,1 \times 10^{-16}$  em 14 iterações. A conservação sobrevive à combinação mais agressiva de funcionalidades. **APROVADO**

## 5.2 C2. Independência de malha e extrapolação de Richardson

$k_{\text{eff}}$  do reator refletido em malhas encaixadas:

| $N$              | 150        | 300        | 600        | 1200       |
|------------------|------------|------------|------------|------------|
| $k_{\text{eff}}$ | 1,27760317 | 1,27757966 | 1,27757377 | 1,27757229 |

Ordem observada em  $k$ : 1,999. Extrapolado de Richardson:  $k_{\text{eff}}^{\infty} = 1,27757180$ ; a malha de  $N = 1200$  difere disso por apenas  $4,9 \times 10^{-7}$  ( $< 0,05$  pcm). **APROVADO**

## 5.3 C3. Invariâncias geométricas

Espelhar o problema 1D (trocar esquerda↔direita): diferença relativa  $4,3 \times 10^{-14}$ . Transpor o problema 2D (rotação de  $90^\circ$ ): diferença  $3,1 \times 10^{-15}$ . Nenhuma assimetria espúria de discretização ou de indexação. **APROVADO**

## 5.4 C4. Sensibilidade à semente do chute inicial

O autovalor usa chute aleatório com semente fixa (42). Com sementes  $\{1, 7, 42, 123, 999\}$ , a dispersão de  $k_{\text{eff}}$  é  $3,8 \times 10^{-14}$  — ordens de grandeza abaixo de  $\text{tol\_k} = 10^{-8}$ . O critério duplo de parada (em  $k$  e na forma do fluxo) é efetivamente robusto ao chute. **APROVADO**

## 5.5 C5. Subcrítico profundo ( $k \approx 0,39$ )

Preocupação levantada *a priori*: os *shifts* fixos do Wielandt ( $\delta = 0,02/0,002$ ) assumem  $k$  bem estimado nas 8 iterações livres. Teste com  $k_{\infty} = 0,4$ : Wielandt converge em 15 iterações ( $k = 0,39058487$ ), *power* em 81, diferença  $6,2 \times 10^{-9}$ . A estratégia de dois estágios se mostrou robusta também longe de  $k = 1$ . **APROVADO**

## 5.6 C6. Malha mínima aceita ( $N_x = 3$ )

Mesmo na malha mais grosseira que a validação permite, o balanço fecha com resíduo  $1,8 \times 10^{-16}$  (a *acurácia* é obviamente baixa, mas a *conservação* — propriedade estrutural do esquema FV — é exata). **APROVADO**

# 6 Frente D — Robustez da camada de entrada

Testes executados contra `main.py` como usuário final. Critério: entrada inválida deve produzir **erro limpo** (mensagem `[ERRO de input]`, sem *traceback*) ou, quando a entrada é tecnicamente válida porém suspeita, um **aviso**.

**Análise das falhas.** Todas as cinco são da mesma família — *tratamento de borda da camada de entrada* — e têm correção localizada:

1. **D2/D4**: o `ValueError` de `malha.py` e o de `int()` não são `ErroDeInput`, então escapam do `except` de `main.py:106`. Converter/encapsular esses erros resolve D2 e D4 de uma vez.
2. **D3**: capturar `json.JSONDecodeError` e `yaml.YAMLError` em `ler_input` e reemitir como `ErroDeInput` com linha/coluna.
3. **D5**: um aviso de uma linha no eco do input (“fonte fixa ignorada em modo autovalor”) basta.

| Teste                                                         | Comportamento observado                                                                                                                              | Veredito         |
|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| D1: $\Sigma_s > \Sigma_t$                                     | Erro limpo com mensagem clara e valores ecoados                                                                                                      | <b>APROVADO</b>  |
| D2: regiões não cobrem o domínio                              | <code>ValueError</code> com <i>traceback</i> cru (a mensagem em si é boa, mas vaza pilha: <code>main.py</code> só captura <code>ErroDeInput</code> ) | <b>REPROVADO</b> |
| D3: JSON malformado                                           | <code>JSONDecodeError</code> com <i>traceback</i> cru                                                                                                | <b>REPROVADO</b> |
| D4: <code>Nx: "quatrocentos"</code>                           | <code>ValueError</code> de <code>int()</code> com <i>traceback</i> cru, sem apontar o campo                                                          | <b>REPROVADO</b> |
| D5: <code>fonte &gt; 0</code> em modo autovalor               | Executa e ignora a fonte <b>silenciosamente</b> (código de saída 0, nenhum aviso)                                                                    | <b>REPROVADO</b> |
| D6: arquivo YAML vazio                                        | Erro limpo (“o input deve ser um mapeamento”)                                                                                                        | <b>APROVADO</b>  |
| D7: interface de material coincidindo com centróide de célula | O resultado depende da <b>ordem</b> em que as regiões são listadas: diferença relativa de 68,7% entre as duas ordens, sem qualquer aviso             | <b>REPROVADO</b> |

Tabela 2: Robustez de entrada *antes das correções*: 3 aprovados (D1, D6 e o já coberto pela suíte), 5 reprovados. Nenhuma falha compromete resultados de entradas válidas. Ver Seção 7 para o estado final.

4. **D7**: é o caso mais sutil e o mais perigoso, porque é *silencioso e fisicamente grande* (68% no caso construído, deliberadamente patológico:  $N = 10$  com interface exatamente sobre um centróide). Mitigações possíveis: detectar interface não alinhada com faces da malha e avisar, ou fazer *snap* das interfaces às faces. O `README` documenta a regra “a última região sobreposição”, mas documentação não substitui aviso em tempo de execução.

## 7 Correções aplicadas

As cinco falhas da frente D foram corrigidas, sem alterar uma linha do núcleo numérico (`montagem.py`, `solver.py`, `balanco.py` intactos):

1. **D2** — `malha.py` passa a lançar `ErroDeInput` (subclasse de `ValueError`, preservando a compatibilidade da suíte de testes) quando as regiões não cobrem o domínio; o erro agora é capturado limpo por `main.py`.
2. **D3** — `ler_input` captura `json.JSONDecodeError` e `yaml.YAMLError` e reemite como `ErroDeInput` com a posição do defeito no arquivo.
3. **D4** — novos validadores `_numero/_inteiro` exigem tipo numérico em `Lx`, `Nx`, `Ly`, `Ny`, apontando o campo ofensor na mensagem.
4. **D5** — em modo autovalor com `fonte > 0`, é emitido `[AVISO]` explícito de que a fonte fixa é ignorada.
5. **D7** — `materiais_1d/materiais_2d` detectam borda de região coincidente com centróide de célula e avisam que a atribuição segue a última região listada, recomendando alinhar a interface a uma face da malha.

Após as correções, a bateria completa foi reexecutada: **21/21 verificações aprovadas** (frentes A–D), suíte própria 18/18, e os 12 casos de `inputs/` executam sem regressão (mesmos resíduos de balanço). Na mesma revisão foram removidos comentários informais do `README` e comentários supérfluos do código e dos arquivos de entrada, mantendo apenas os *docstrings*-título de cada função.

## 8 Veredito

| Frente                                  | Testes  | Aprovados | Situação      |
|-----------------------------------------|---------|-----------|---------------|
| Suíte própria + casos obrigatórios      | 18 + 13 | todos     | sólida        |
| A. Verificação analítica                | 3       | 3/3       | sólida        |
| B. Física de reatores                   | 5       | 5/5       | sólida        |
| C. Consistência numérica                | 6       | 6/6       | sólida        |
| D. Robustez de entrada (antes)          | 7       | 2/7       | <b>frágil</b> |
| D. Robustez de entrada (após correções) | 7       | 7/7       | sólida        |

**Núcleo numérico e física: aprovados sem ressalvas.** O código demonstra ordem 2 verificada contra dois benchmarks analíticos independentes (fonte fixa e autovalor — este último ausente da suíte original e adicionado por esta avaliação), conservação de nêutrons em precisão de máquina mesmo no pior caso combinado, sensibilidades com o sinal e a magnitude corretos, invariâncias exatas e solvers consistentes entre si inclusive em regime subcrítico profundo.

**Camada de entrada: falhas identificadas e corrigidas.** As cinco falhas D2–D5 e D7 eram todas de *experiência de erro*, não de física; nenhuma produzia resultado errado a partir de entrada válida e bem posta. Após as correções da Seção 7, os sete testes da frente D passam.

**Pendências documentais:** o `README` referencia notebooks (`difusao_opcoes_1_e_3.ipynb`) ausentes do repositório e contém comentários informais que não condizem com a qualidade do restante do trabalho; as dependências exigem `venv` não documentado explicitamente.

## A Reprodutibilidade

- Suíte própria: `.venv/bin/python -m pytest tests/ -v` (saída em `relatorio/dados/pytest_saida.txt`)
- Casos obrigatórios: `.venv/bin/python main.py inputs/caso*.yaml` (`relatorio/dados/casos_obrigat`)
- Bateria A–D: `.venv/bin/python relatorio/testes_professor.py` (resultados completos em `relatorio/dados/resultados.json`);
- Inputs patológicos gerados: `relatorio/dados/input_*.yaml/.json`.